

# **Primjena paterna ponašanja u sistemu Smart Hair Salon**

## **Uvod**

Sistem Smart Hair Salon predstavlja web platformu za upravljanje uslugama frizerskog salona i online prodajom proizvoda za njegu kose. Korisnicima je omogućeno rezervisanje termina, kupovina proizvoda, pregled salona na mapi, kao i praćenje statusa svojih rezervacija i narudžbi. Kako sistem sadrži više međusobno povezanih funkcionalnosti i različite vrste korisnika, javlja se potreba za primjenom obrazaca ponašanja (Behavioral Design Patterns) radi poboljšanja fleksibilnosti, održavanja i proširivosti sistema.

## **Observer Pattern**

### **Problem**

Jedna od ključnih funkcionalnosti sistema jeste obavještavanje korisnika o statusu rezervacije termina ili narudžbe proizvoda. Nakon što zaposlenik obradi zahtjev, korisnik treba biti obaviješten o tome da li je rezervacija ili narudžba prihvaćena ili odbijena.

Bez korištenja odgovarajućeg obrasca ponašanja, modul za obradu rezervacija morao bi direktno komunicirati sa svim komponentama koje šalju obavijesti. Takvo rješenje povećava međusobnu zavisnost komponenti i otežava buduća proširenja sistema.

### **Predloženo rješenje**

Observer obrazac omogućava da objekti koji žele biti obaviješteni o promjenama stanja budu registrovani kao posmatrači (observers). Kada se status rezervacije ili narudžbe promijeni, sistem automatski obavještava sve registrovane posmatrače. U sistemu Smart Hair Salon objekat rezervacije ili narudžbe može predstavljati subjekt (Subject), dok komponente za slanje obavijesti predstavljaju posmatrače (Observers).

### **Primjena u sistemu**

Kada zaposlenik prihvati ili odbije rezervaciju:

- status rezervacije se mijenja,
- aktivira se mehanizam obavještavanja,
- korisnik dobija odgovarajuću notifikaciju.

Na isti način moguće je dodati nove vrste obavijesti bez izmjene postojećeg koda, što povećava proširivost sistema.

### **Prednosti**

- smanjenje povezanosti između komponenti sistema,
- jednostavno dodavanje novih vrsta obavijesti,
- bolja podrška za asinhronne operacije,
- lakše održavanje sistema.

## **State Pattern**

### **Problem**

Rezervacije i narudžbe tokom svog životnog ciklusa prolaze kroz više stanja. Na primjer, rezervacija može biti:

- na čekanju, • prihvaćena, • odbijena, • završena.

Slično tome, narudžba proizvoda može prolaziti kroz različite faze obrade.

Implementacija velikog broja uslovnih grananja (if-else ili switch struktura) za upravljanje svim mogućim stanjima može dovesti do složenog i teško održivog koda.

### **Predloženo rješenje**

State obrazac omogućava da se svako stanje sistema predstavi posebnom klasom koja definiše dozvoljeno ponašanje u tom stanju.

Na taj način logika vezana za pojedino stanje nije koncentrisana na jednom mjestu, već je raspoređena u odgovarajuće klase.

### **Primjena u sistemu**

Za rezervaciju termina mogu se definisati sljedeća stanja:

- PendingReservationState,
- AcceptedReservationState,
- RejectedReservationState,
- CompletedReservationState.

Svako stanje definiše koje su naredne dozvoljene promjene statusa.

Na primjer:

- rezervacija u stanju "Na čekanju" može preći u stanje "Prihvaćena" ili "Odbijena",
- rezervacija u stanju "Prihvaćena" može preći u stanje "Završena".

Na isti način moguće je modelirati i životni ciklus narudžbe proizvoda.

### **Prednosti**

- jednostavnije upravljanje životnim ciklusom rezervacija i narudžbi,
- uklanjanje velikog broja uslovnih grananja,
- lakše dodavanje novih stanja u budućnosti,
- bolja organizacija poslovne logike.

## **Strategy Pattern**

### **Problem**

Sistem Smart Hair Salon sadrži više različitih načina obrade rezervacija i narudžbi. Na primjer, provjera dostupnosti termina može se vršiti prema različitim pravilima:

- prema radnom vremenu salona,
- prema zauzetosti zaposlenika,
- prema vrsti usluge,
- prema trajanju tretmana.

Ako bi se svi algoritmi implementirali unutar jedne klase korištenjem velikog broja uslovnih grananja, kod bi postao složen i teško proširiv.

### **Predloženo rješenje**

Strategy obrazac omogućava definisanje više različitih algoritama kroz odvojene klase, pri čemu se konkretna strategija bira tokom izvršavanja programa.

U sistemu Smart Hair Salon moguće je definisati različite strategije za provjeru dostupnosti termina ili obračun cijena usluga.

### **Primjena u sistemu**

Prilikom rezervacije termina:

- sistem bira odgovarajuću strategiju provjere dostupnosti,
- provjerava da li termin zadovoljava pravila salona,
- vraća rezultat korisniku.

Moguće strategije:

- WorkingHoursStrategy,
- EmployeeAvailabilityStrategy,
- ServiceDurationStrategy.

## **Prednosti**

- jednostavno dodavanje novih algoritama,
- smanjenje kompleksnosti koda,
- lakše održavanje sistema,
- fleksibilniji način obrade podataka.

## **Command Pattern**

### **Problem**

U sistemu postoji veliki broj akcija koje korisnici i zaposlenici izvršavaju:

- rezervacija termina,
- otkazivanje rezervacije,
- potvrda narudžbe,
- odbijanje zahtjeva,
- dodavanje proizvoda.

Direktno pozivanje svih operacija između komponenti povećava povezanost sistema i otežava praćenje izvršenih akcija.

### **Predloženo rješenje**

Command obrazac omogućava da se svaka akcija predstavi kao poseban objekat. Na taj način moguće je odvojiti pošiljaoca zahtjeva od izvršioca operacije. Svaka operacija u sistemu može biti predstavljena posebnom komandnom klasom.

### **Primjena u sistemu**

Moguće komande:

- CreateReservationCommand,
- CancelReservationCommand,
- AcceptOrderCommand,
- RejectReservationCommand.

Kada korisnik izvrši određenu akciju:

- kreira se odgovarajuća komanda,
- komanda se proslijeđuje izvršiocu,
- sistem izvršava operaciju.

Prednosti

- smanjena povezanost komponenti,
- jednostavno dodavanje novih akcija,
- mogućnost evidencije izvršenih operacija,
- bolja organizacija poslovne logike.

## **Mediator Pattern**

### **Problem**

Komponente sistema poput korisnika, zaposlenika, rezervacija, narudžbi i obavijesti međusobno intenzivno komuniciraju. Direktna komunikacija između svih komponenti povećava složenost sistema.

### **Predloženo rješenje**

Mediator obrazac uvodi centralnu komponentu koja koordinira komunikaciju između objekata.

Na taj način objekti ne komuniciraju direktno međusobno, već preko posrednika (Mediator).

### **Primjena u sistemu**

U sistemu Smart Hair Salon:

- korisnik šalje zahtjev za rezervaciju,
- mediator prosljeđuje zahtjev zaposleniku,
- zaposlenik obrađuje zahtjev,
- mediator aktivira slanje obavijesti korisniku.

Centralna klasa može biti:

- SalonMediator.

### **Prednosti**

- smanjenje međusobne zavisnosti komponenti,
- centralizovana komunikacija,
- lakše održavanje i proširenje sistema,
- preglednija arhitektura.

# Template Method Pattern

## Problem

Proces obrade rezervacije i proces obrade narudžbe imaju slične korake:

- validacija podataka,
- provjera dostupnosti,
- obrada zahtjeva,
- ažuriranje statusa,
- slanje obavijesti.

Implementacija ovih procesa bez zajedničke strukture dovodi do ponavljanja koda.

## Predloženo rješenje

Template Method obrazac definiše osnovni tok algoritma u baznoj klasi, dok pojedini koraci mogu biti redefinisani u izvedenim klasama.

## Primjena u sistemu

Moguće klase:

- ReservationProcessingTemplate,
- OrderProcessingTemplate.

Oba procesa koriste iste osnovne korake, ali imaju različitu implementaciju pojedinih faza.

## Prednosti

- smanjenje dupliciranja koda,
- standardizovan proces obrade,
- lakše održavanje sistema,
- jednostavnije proširenje funkcionalnosti.

# Chain of Responsibility Pattern

## Problem

Prilikom obrade rezervacija i narudžbi potrebno je izvršiti više provjera:

- validacija podataka,
- provjera dostupnosti termina,
- provjera zaliha proizvoda,
- provjera statusa korisnika.

Ako bi jedna komponenta obavljala sve provjere, kod bi postao nepregledan i teško održiv.

## Predloženo rješenje

Chain of Responsibility obrazac omogućava povezivanje više objekata koji sukcesivno obrađuju zahtjev.

Svaki objekat izvršava svoju provjeru i prosljeđuje zahtjev narednom objektu u lancu.

## Primjena u sistemu

Lanac obrade može sadržavati:

- ValidationHandler,
- AvailabilityHandler,
- StockHandler,
- PaymentHandler.

Zahtjev prolazi kroz sve faze prije konačne potvrde.

## Prednosti

- jasna podjela odgovornosti,
- lakše dodavanje novih provjera,
- fleksibilnija obrada zahtjeva,
- pregledniji kod.

# **Iterator Pattern**

## **Problem**

Sistem upravlja velikim brojem rezervacija, proizvoda, korisnika i narudžbi. Potrebno je omogućiti jednostavan pregled i filtriranje podataka bez otkrivanja interne strukture kolekcija.

## **Predloženo rješenje**

Iterator obrazac omogućava sekvencijalni pristup elementima kolekcije bez direktnog pristupa njenoj implementaciji.

## **Primjena u sistemu**

Iterator se može koristiti za:

- pregled svih rezervacija,
- pregled proizvoda u webshopu,
- prikaz narudžbi zaposleniku,
- filtriranje dostupnih termina.

Moguće klase:

- ReservationIterator,
- ProductIterator,
- OrderIterator.

## **Prednosti**

- jednostavniji pristup kolekcijama podataka,
- sakrivena interna implementacija kolekcija,
- lakše filtriranje i pretraga podataka,
- pregledniji kod.